SpecPilot：把「改进了没有」变成脚本能算的量

> 状态：完成
> 信息截止：2026-11-05
> 来源数：3（论文 / 仓库 / 项目页） | 证据数：81 | 主张数：33（29 高置信 / 1 争议）
> 审计对象：arXiv 2611.02501 v1（2026-11-02）；GitHub `StargazerX0/SpecPilot` @ `78c42af`

1. 执行摘要

SpecPilot 是 UC Berkeley / Princeton / Stanford / NVIDIA 的一篇系统论文：它让一组 Claude Opus 5.5 agent 在 SGLang 上自动改投机解码器，十三轮之后把单请求加速从 stock 的 4.46× 推到 10.87×、批处理从 1.94× 推到 3.93×，全程不训练、不损精度 `[c-019]`。

读完论文与源码，本归档的判断是：**它的贡献不在那几个机制，而在把「改进了没有」拆成了三个可被脚本判决的量。**

- **该往哪看**：成本模型把「离上界还差多少」分解成三个因子——树能多收多少 token、kernel 离 byte floor 多远、profile 之外还有多少开销 `[c-003]`。每轮重算，取最大的那个作为下一轮方向。三轮里指向变了两次：树 → 权重字节 → 「一步的单位」`[c-004]`。
- **算不算改进**：agent 只能调控制器脚本、不能改；晋级要求**增益的 95% 下界**同时超过 1% 与 2×（stock 对 stock 的噪声），输出与当前最优不同时门槛抬到约 3% `[c-002]`。H100 单请求噪声 <0.1%，而典型改动只有 1.06–1.08×（§3.2）——门槛就是为这个量级设计的。
- **还无不无损**：论文拒绝用「逐位相同」（会误杀）与「准确率不降」（会漏放）判定，改用 teacher-forced 重打分 + 两条统计拒绝规则 `[c-006]` `[c-008]`；并用注入故障实验证明前者确实分不开、后者确实分得开 `[c-009]`。

三条里最有信息量的是第三条，因为它给出了**判据失效的直接证据**；最有信息量的负结果是第一条的反面：没有成本模型的单 agent 循环，三轮只到 1.34×，而 SpecPilot 是 2.13× `[c-005]`。

源码侧有一个必须说清的落差，而且它有两面。 仓库交付的是六个 setting 的**结果引擎**——每个 final = stock 树 + 该 setting 的 `files/` 覆盖（不删任何文件）+ 46–66 个 `SPECPILOT_*` 开关 `[c-026]`。

一面是机制确实在：333 个 overlay 文件里 183 个是 `specpilot_*` / `dflash_d*` 实现，逐 setting 25–37 个。W12 的 12KB 嵌套平面、Q pass 的零额外显存读取、树验证的三段 BF16 WY kernel、prompt admission 的强制链，都能在源码里逐行对上论文 `[c-032]`。连故障注入的钩子都留了——`SPECPILOT_D8R_TLV_QNOISE` 按概率把 Q 的预测换掉，专门制造链失配 `[c-033]`。

另一面是判据确实不在：承担判决的三样东西（控制器、记忆库、token forcing 的失配审计）在公开树里找不到 `[c-027]`。这不是「grep 不到」的印象：**仓库自己的代码引用了 22 条路径，其中 10 条不存在**，缺失集中在 `research/knowledge/`（7 篇）与 `research/validation/`（2 个）`[c-031]`。

两面合起来才是准确的形状：**造故障的装置留了，判定故障的脚本没留。**裁剪口正好在「循环怎么知道改对了」这一侧。

一句话：**机制是可以替换的产物，判据才是可迁移的部分；而这次公开的是完整的产物，加半个判据。**

2. 范围与方法

2.1 与站内三篇相邻归档的分工

归档	关注点	本文的关系
`kernel-agent`	agent 循环的五个环节（观测/动作/预算/恢复/评测）	那篇问「系统怎么把 kernel 迭代到可用」，本文问「循环怎么敢说改进了」
`cuda-kernel-rl`	奖励设计	本文不涉及训练，只涉及评测与晋级
`pd-disagg`	部署形态的收益边界	本文的边界是「同一机制在不同 regime 下反号」

三篇都不涉及投机解码，也都没有把「判定规则本身」当作研究对象。

2.2 三份来源怎么分工

来源	层	用途	不能用来做什么
arXiv 2611.02501 全文	paper / primary	机制、数字、判据定义的全部原始出处	不能证明代码存在
GitHub 仓库 @ `78c42af`	code / primary	交付边界、开关、构建方式、脚本职责	不能证明论文的循环被交付
项目页	page / secondary	交叉核对措辞（晋级门槛、批处理反号）	转述，无新观测

为什么仓库只值 3 个来源、却给了 81 条证据里的 33 条（paper 46 / code 33 / page 2，见 `analysis.json` 的 `evidence_by_layer`）：仓库不是「又一篇文献」，是**被审计的对象**。它的证据形态也不同——不是「作者说了什么」，而是「树里有什么、代码引用了什么、缺了什么」。

2.3 证据是怎么取的（两道机器闸门）

1. **E1（原文子串）**：81 条 quote 全部由 `build_specpilot.py` 的 `grab()` 从快照实取，不是手写。校验器逐条比对，结果 `e1_checked=81, e1_unverifiable=0`。
2. **引用闭合**：33 条主张引用的证据全部存在，且 81 条证据**没有一条零引用**（`evidence_dangling=[]`）。

取证据的过程本身踩了两个坑，都记在 §6。

2.4 仓库侧的数字全部由脚本产出

`audit_repo.py` 数文件与开关，`audit_dangling.py` 查悬空引用，两者输出 `repo_audit.json` / `dangling_audit.json`；`make_analysis.py` 汇总成 `analysis.json`。**正文里没有一个仓库数字是手抄的**——包括「22 条引用里 10 条缺失」这句。

3. 循环：三个判决点

3.1 五阶段与一次否决 [c-001]

阶段	谁做	产出	判定
Phase 0	成本模型	三个因子，指出最大缺口	无（只测量）
Phase 1	master agent 开方向（每条一块 GPU），sub-agent 实现	patch（默认关闭即 stock）	先过正确性检查与 token forcing，再计时
Phase 2	integration sub-agent	叠加若干单独不够门槛的 patch	同一把尺子量整个栈
Phase 3	kernel sub-agent	修离 byte floor 最远的 kernel	同上
Phase 4	控制器	**sealed final**（未见过的请求）	token forcing 不过即整轮作废

关键设计是 patch 默认关闭：`code changes behind options whose defaults reproduce stock` [e-009]。于是「未晋级的改动」不会污染基线，也不需要回滚。

Phase 4 的 sealed final 是整个循环的信任来源：开发请求 agent 看得到、可以针对；sealed 请求看不到。**主表报的是 sealed 增益**，对 AR 的速度比另算 [c-020]。

3.2 晋级条件：把噪声写进门槛 [c-002]

```
晋级 ⇔ 增益的 95% 下界 > 1% + 2 × (stock vs stock 噪声)
       输出与当前最优不一致时，门槛抬到 ~3%
```

三个细节值得单独说：

1. **下界而非点估计。** 点估计 3% 但方差大的 patch 会被拒。
2. **噪声不是常数，是每轮实测的。** 论文给的 H100 单请求值 <0.1%，于是 1% + 2×0.1% ≈ 1.2%；批处理下噪声更大，门槛自动抬高。
3. **不一致要加价。** 输出与当前最优不同意味着语义可能变了，于是要求三倍增益。

配套的是单条改动的量级：W12 的首次晋级是 1.076×，改内核后 1.063× [e-046]。**门槛的刻度与改进的刻度是匹配的**——这一点比阈值大小本身更重要。

3.3 方向关闭：先测再关 [e-010]

```
只有在「实现了该机制并测量」之后才关闭一个方向，
除非实现它需要超过 10 GPU-分钟。
```

这条规则的产物是附录 F 那张表 [c-022]：11-bit 权重解码慢 3×、48 节点树 −4.3%、超过 32 节点后每节点值 0.063ms 却要 0.17ms、嵌套 Q 视图每减 1/8 字节掉约 0.01 的 top-1 一致率。**负结果带测量依据**，这是本领域最稀缺的东西。

4. 成本模型：让搜索不被最显眼的瓶颈吸住

4.1 从 S=τ/c 到三分之积

速度比写成 `S(N)=τ(N)/c(N)`，其中 `c(N)=T_step(N)/T_AR` `[e-004]`。ceiling 是「一次 draft、一次 exact pass、每类 kernel 跑到 byte floor」时的上界，因子分解为 `[e-008]`：

```
ceiling = (τ_best / τ) · (T_prof / T_floor) · (T_step / T_prof)
          └ 树 ┘    └── kernel ──┘    └ profile 外 ┘
```

byte floor 取「60% 峰值稠密算力的 FLOP 时间」与「90% 峰值带宽的字节时间」的较大者 `[e-007]`——即**没有 kernel 能低于这个时间**，所以差距可量化。

4.2 三轮里指向变了两次 [c-004]

轮	最大因子	动作	结果
一	τ（树）	best-first 树，32 节点	184 → 270 tok/s
二	T_prof/T_floor（步成本）	W12 权重流 + certified LM head	270 → 更高（论文未逐轮给数）
三	两个都接近上限	**打破 ceiling 自己的假设**（TLV）	→ 416 tok/s

第三轮是全文最有信息量的一步。当两个因子都贴近上限时，剩下的增益不在因子内，而在**因子的定义里**：ceiling 假定「一次 draft 配一次 exact pass」，TLV 让一次 exact pass 服务两次 draft，于是 τ 的定义从「每步」变成「每次 exact pass」，上界随之上移 `[c-025]`。

论文在 PRO 6000 上留下了这个推断的直接证据：最终 2.44× **超过**它自己估计的 2.12× ceiling `[e-035]`。

4.3 有没有成本模型的对照 [c-005]

	三轮后增益	三轮里碰了什么
单 agent（无成本模型）	1.34×	24 节点树、更长 drafter block、更便宜的 drafter 权重——全在「每步 token 数」上
SpecPilot	2.13×	树 → 权重字节 → 一步的单位

对照的问题不是不努力，而是**它优化了最显眼的因子，始终没碰占步时 80% 的权重读取** `[e-006]`。这是本归档认为「成本模型是主要贡献」的最强证据——但它只有**一个 setting、一次运行**，所以只支持「在这一个 setting 上有效」。

5. 「无损」怎么被验证

5.1 定义先降一格 [c-006]

```
无损 ⟺ 每个被接受的 token 都落在一次精确权重 pass 验证过的路径上，
       且等于该 pass 的 argmax
```

注意它**不要求**与 AR 逐位相同：浮点归约顺序（GEMM tiling、tree attention）会让近 tie 翻转，stock 自己就这样偏离 AR，首次偏离处两 token 的 log-prob 差 ≤ 0.232 nats [e-014]。

这一步是全篇的地基：不先把「无损」降格为「相对精确 pass 的无损」，后面的统计判据就没有立足点。

5.2 token forcing 的构造 [c-007]

判据要能跑起来，先得有一个「独立裁判」。做法是把系统自己吐出的 token 序列**强制喂回** stock 引擎，用 FP32 的 LM head 逐位置重打分，再比对「系统当时提交的 token」是否等于「裁判给出的 argmax」[e-012]：

```
输出 token 序列 ──▶ stock 引擎（teacher forcing）──▶ FP32 LM head ──▶ 逐位置 argmax
                          ▲                                              |
                          └──── 比对：提交的 token == argmax ? ──────────┘
                                                  |否
                                                  └─▶ mismatch，记其 margin（到 argmax 的 log-prob 差）
```

三个设计要点：

1. **裁判是 stock，不是 AR。** 这样比的是「实现有没有引入新错误」，而不是「投机解码该不该和 AR 完全一致」——后者本来就不该成立。
2. **FP32 LM head 是必要的。** 用低精度 head 打分，会把 head 自己的舍入误差算进系统的账。
3. **stock 自身有 0.72% 的失配**，几乎全是 kernel 归约造成的近 tie [e-012]。判据不能要求 0，只能要求「不显著高于这个本底」。

第 3 点是整个判据设计的枢纽：**基线不干净，所以阈值必须由基线的分布定出来**，而不是由「理想无损」定出来。δ 的定法（§5.3）正是这条原则的产物。

5.3 两条拒绝规则 [c-008]

规则	统计量	阈值	管什么
rate rule	失配率与 stock 之差	> 0.5 个百分点（95% 置信）	整体分布漂移
event rule	margin > 2δ 的失配数	二项检验显著多于 stock	尾部系统性错误

两个参数都不是拍的：

- **δ**（近 tie 阈值）= stock 首次偏离 AR 时的最大 margin，向上取整到 1/8 的倍数 [e-012]。把「基线自己也不干净」变成阈值的一部分，而不是要求基线为 0。
- **stock 自身的失配率 0.72%**，几乎全是近 tie [e-012]。判据的分辨率就建立在「基线有本底噪声」这个事实上。

事件可以被「解释掉」，但只有三种解释：stock 在自身路径上能复现、margin 与 stock 相当、或打分器 argmax 随 prefill 长度改变 [e-013]。

5.4 为什么准确率不够 [c-009]

给每第 7 个 vocab id 在 2048 token 后加 2 的 logit bias，造出「看起来正常、实际有偏」的故障系统 [e-015]：

系统	失配率	未解释的大 margin 事件	准确率变化
无损 Qwen3.8-27B	0.73%	0 of 200	−0.3 点 [−1.6, +0.7]
无损 Qwen3.5-4B	0.52%	0 of 200	—
故障 27B	2.0%	192 of 200	−1.6 [−3.6, +0.3]
故障 4B	2.4%	191 of 200	—

准确率区间重叠（−0.3 与 −1.6 的置信区间覆盖 0），而失配率把它们分开约 3×。这是全文说服力最强的实验——它同时给出了判据的假阴性（无损系统贴近 stock）与假阳性（故障系统被抓住）两侧。

5.5 判据仍然漏掉什么

- **只对 stock 比较，不对 AR 比较。** 若 stock 自己错，SpecPilot 跟着错也不会被发现。
- **统计判据需要样本量。** 每条请求 2048 token；更短或更同质的负载下，0.5 点的分辨率未必够。
- **「解释掉」依赖人工判断。** 三种解释是穷举还是举例，论文没给证明。

6. 机制：四个零件与它们的代价结构

6.1 W12：无损 12-bit 权重格式 [c-010] [c-011]

每个输出通道取覆盖最多权重的 16-binade 窗口，权重存 sign + 4-bit 指数偏移 + 7-bit 尾数；窗外、零、denormal 走稀疏 escape 表精确还原 [e-017]。实测 12.01–12.02 bits/weight，Qwen3.8-27B 的转换层 44.67 → 33.58 GB，加载时逐张量按位比对。

它的收益一半来自格式、一半来自 kernel：GEMM 在寄存器里解码，12KB 记录对应 128×64 tile，stream-K 固定归约顺序保证确定性 [e-018]。首版 kernel 只有 cuBLAS 的 0.80–0.89 时间，是被 kernel 子 agent 调上去的 [e-042]。

W12 还能脱离投机解码：纯 AR 解码上单独开启即得 1.19–1.28×（1 请求）/ 1.13–1.20×（32 请求）[c-012]。这让它从「SpecPilot 的零件」变成可独立采纳的工程结论。

6.2 best-first 树：换打分，不换骨架 [c-013]

按父节点给的 log-softmax 打分（不是 DDTree 的边缘分），32 节点、深 15、drafter 跑 16 位置 [e-020]。exact replay 下每步接受 token 4.25 → 5.38（8 位置 / 32 节点）→ 5.85（16 位置）[e-043]。

两处边界很清楚：16 节点预算下与 EAGLE-2 式 top-4 打平（5.03 vs 5.02），说明**继续加宽收益有限**；而批处理下 24 节点树 **−14.9%** [c-021]。

6.3 混合目标的前提：不存快照 [c-014]

Qwen3.8-27B 的 64 层里 48 层是 GDN。树验证若为每节点存一份递归状态快照，显存与带宽会把收益吃掉，故改为沿根路径从已提交状态重算；WY kernel 把 FP32 状态拆成三个 BF16 部分表示全部 24 位有效位，在 tensor core 上一次解出所有路径，与 path kernel 差在一个 BF16 ulp 内 [e-021]。

两者只改归约顺序，故 exact pass 仍精确——这是能让「树」在混合架构上成立的前提。

6.4 TLV：唯一让 ceiling 失效的机制 [c-015] [c-016]

阶段	耗时	说明
两次 draft	1.05ms × 2	Q pass 之后从链 C 末尾再 draft
Q pass	11.5ms	在 W12 已嵌套的 8-bit 视图上跑，只提议不提交
exact pass	14.9ms	同时验证 C 与挂在它下面的子树
胶水	1.6ms	
一轮	**~30ms**	换 **11.67** vs 6.16 token / exact pass

无损性来自「**只有 exact pass 提交 token**」：Q pass 不更新 KV 与递归状态，故它猜错只影响效率、不影响正确性 [c-015]。Q 与 exact 的 top-1 一致率约 98%（嵌套 8-bit 视图 0.983）[e-024]。

代价结构解释了它的反号：Q pass 占一步的 0.80–0.88 时，一轮必须接受接近两倍的 token 才不亏。单请求下权重流把 Q pass 藏住了，批处理下它按行累加——于是 **BF16 批处理下 TLV −13%** [c-021]。

6.5 批处理：换成「少花」[c-017] [c-018]

单请求侧的机制都是「多花验证工作」，批处理侧全部反过来：

机制	做法	增益
共享节点预算	按路径 log-prob 在请求间分配固定总预算；best-first 前缀闭包保证每份仍是合法树	DSpark 上多收 6–14%
每请求 drafter block	按近期接受率与置信度选 16/12/8 位置，压在 256 行内	1.040×
prompt 准入	只在调度器本来要压住时才并流	+3.5%（**未配对**）

第三条作者自己标了 unpaired，故该主张降为 medium [c-018]；而「准入所有短 prompt」是负收益 [e-038]。

7. 增益与口径

7.1 主结果 [c-019]

Setting	对 AR	对 stock（sealed）
H100 单请求 DFlash2	8.86×	2.13× [2.07, 2.19]
H100 单请求 DSpark	9.41×	2.38× [2.33, 2.43]
H100 批处理 DFlash2	3.90×	1.85× [1.79, 1.91]
H100 批处理 DSpark	3.93×	1.94× [1.91, 1.97]
PRO 6000 单请求 Qwen3.8-27B	10.87×	2.44× [2.37, 2.50]
PRO 6000 单请求 Muse-30B	10.63×	2.21× [2.15, 2.26]

7.2 三处不可比 [c-020]

1. **「对 AR」与「对 stock」不是一回事。** 10.87× 与 2.44× 是同一个 setting 的两个口径。混用会把增益夸大约 4.5 倍。
2. **τ 在 TLV 下改了定义。** 从「每步接受的 token」变成「每次 exact pass 接受的 token」，故 τ 可以超过 16 的 draft block（README 复现表里 13.08 就是这个）。读成「一步接受 13 个 token」会错。
3. **bench 与 sealed 差 −2%~+6%。** 主表是交错的 AR/stock/SpecPilot 运行（开发请求），sealed 是对 stock、未见请求、两次配对运行。

7.3 另一处需要标注的：复现表是作者自跑 [c-026]

README 给 4 个 H100 setting 的复现数字（3.96/8.91、3.77/9.48、2.05/3.96、1.92/3.91）与 95% bootstrap CI，但是**作者在新 clone + 新环境里自跑**，不是第三方独立复现 [e-103]。它证明「仓库能跑出接近的数」，不证明「任何人能跑出」。

8. 边界：同一机制为什么会反号

8.1 反号的方向与量级 [c-021]

机制	单请求	批处理
宽树（24 节点）	增益	**−14.9%**（FP8 目标）
长 drafter block（24 位置）	增益	**−9.9%**（BF16 Qwen3.8-27B）
TLV	增益	**−13%**（BF16）

原因是同一个：这些机制都**多花验证工作**。单请求时权重流把额外工作藏住了（"verifying additional tokens adds little extra latency"），批处理时它按行累加 [e-037]。

这条边界比任何单个增益都更有用：它给出「什么条件下该关掉哪个机制」的判据。项目页也写了同样的反号，可交叉核对 [e-049]。

8.2 循环本身的成本 [c-023]

项	值
一轮	7–20 小时 / 20–86 GPU-小时
十三轮合计	164 小时 / 530 GPU-小时
代码改动	486 个
晋级	54 个（约 11%，改动级）

按本文口径，一个 setting 的完整结果需要 1–2 轮。**这套方法的门槛不是算力，是耐心**：530 GPU-小时换六个 setting，而其中很多轮的方向最后被关掉了。

8.3 记忆能迁移什么 [c-024]

新部署	复用什么	结果
DSpark	树 + W12 + TLV 三条条目	一次晋级 2.110×
PRO 6000	仅 H100 的树与 W12（W12 改 mma.sync、树改 Triton attention）	sealed 1.92×
PRO 6000 + TLV/split-KV	再加两条	2.44×（超 ceiling）
EAGLE-3（顺序 drafter）	W12（MLP）+ prompt 准入	首轮 2.67× → 3.11×

保留规则很硬：**条目只在「套回 stock 能逐字重建源系统代码」时才保留** `[e-016]`；且「复用条目不算新发现」——否则同一份增益会被重复计入。

9. 源码审计：交付了什么、没交付什么

9.1 仓库的规模与结构（全部由脚本数）

项	值	来源
文件总数	3098	`audit_repo.py`
`.py`	2023（其中 `python/` 是 SGLang 裁剪副本）	同上
`.cu` / `.cuh`	62 / 200	同上
setting 下的 `.py`	316 个文件，**去重后 145 个 / 145,644 行**	同上
不同 `SPECPILOT_*` 开关	272	同上

316 → 145 这个去重很关键：同一份 overlay 文件在六个 setting 里重复出现，直接数会把重复算成原创。

六个 setting 的配置：

Setting	final	block	开关	overlay 文件	`specpilot_*` 实现	论文 sealed 增益
h100_c1 dflash2	`int-015-tlvcbn32`	32	46	49	25	2.13×
h100_c1 dspark	`d6-001-qcopy`	32	51	53	28	2.38×
h100_batch dflash2	`int-210-full`	16	61	55	31	1.85×
h100_batch dspark	`d1-002-ldu`	16	66	58	33	1.94×
pro6000 muse	`int-003`	32	37	64	37	2.21×
pro6000 qwen	`k-001-d3qpdl`	32	51	54	29	2.44×

最后一列是这一轮补上的。 它由 `audit_repo.py` 按文件名认定（`specpilot_*` / `dflash_d*`），不靠内容猜测；六个 setting 合计 183 个实现文件。

9.2 交付物是什么 [c-026]

```
final = stock 树（SGLang 裁剪 + dflash_lab 插件）
      + settings/<setting>/files/    （逐文件覆盖，没有任何 final 删文件）
      + config.json 的 final.env / block_size / launch_args
```

这一点由两处独立坐实：`build_engine.py` 的 final 分支只有 `overlay(...)` 一个动作 `[e-118]`，README 也写「no final deletes a file」`[e-121]`。

所以 patch 是「加」而不是「改语义」：默认关闭即 stock 行为，晋级的机制靠环境开关打开 `[e-107]`。这也解释了为什么仓库能同时装下六个互相矛盾的 final——它们不共享一个二进制。

9.3 机制是源码，不是伪代码 [c-032]

这一节是本轮补上的，因为第一版审计有个**范围错误**：脚本把 `python/sglang` 一律当 vendored 排除，而 `settings/<setting>/files/python/sglang/...` 恰好就是补丁本体。333 个 overlay 文件被一起排掉了，「机制在不在代码里」这个问题根本没被真正问过。

修正后逐条核对，论文描述在源码里都能定位：

论文的说法	代码里的对应	证据

论文的说法	代码里的对应	证据
W12：12KB 记录 = 嵌套 Q 区 + B 区	`NESTED PLANES: per (128-row tile, 64-k chunk) a 12 KB record = Q region [8 warps][1 KB] + B region [8 warps][512 B]`	`[e-127]`
Q pass 不额外占显存	「streams only the Q regions … an approximate copy at zero extra memory」	`[e-128]`
树验证不存每节点 fp32 状态	`snapshot-free GDN tree verify … Verify writes no per-node fp32 state`	`[e-132]`
WY kernel 用三段 BF16 凑 24 位	`fp32 state as hi + mid + lo bf16 terms (24 mantissa bits)` + 三个 `tl.dot`	`[e-131]`
prompt admission：短 prompt 并进 decode 批	`prefill as a teacher-forced verify chain … skips the extend forward`	`[e-133]` `[e-134]`
注入故障实验	`SPECPILOT_D8R_TLV_QNOISE` + `perturb_pred()`	`[e-129]` `[e-130]`

两处细节值得单独看：

- `q_pass()` **是零成本的近似视图**，不是第二份权重。它只读 Q 区（1B/weight），B 区的半字节根本不读——这就是论文说「不额外占显存」的字面实现 `[e-128]`。码率账因此对得上：精确 pass 1.5B/weight，Q pass 1B/weight。
- `perturb_pred()` **是个 test hook**，注释直白写着 `test only`，但它被 `dflash_worker_v2.py` 的生产路径调用，由环境变量控制 `[e-130]`。论文的注入故障实验（§6.2）用的应该就是这个钩子。

机制侧是完整的：六个 setting 各有 25–37 个实现文件，论文的四个零件（W12、树、TLV、批处理预算）全在。**这不改变 c-027 的结论**——缺的是判定侧，不是机制侧。

9.4 没交付的三样 `[c-027]` `[c-031]`

组件	论文里干什么	公开树里的状态
控制器	配对运行、晋级判定、sealed final	名为 `cudaslot.py` 的文件是 **CUDA 初始化信号量**，作者自注「只做调度，不参与测量」`[e-115]`
结构化记忆库	140 条条目、最多 12 个前置	「memory」只出现在 README 文字里
token forcing 的失配审计	失配率 + margin + 两条规则	`compare.py` 只报「identical outputs same/tot」`[e-111]`；**造故障的钩子在（§9.3），判故障的脚本不在**

但真正硬的证据不是这些检索空白，而是仓库自证的裁剪口 `[c-031]`：

```
仓库代码引用了 22 条路径 → 10 条不存在
  research/knowledge/    缺 7（方法与搜索空间文档）
  research/validation/   缺 2（测试与预言机）
  research/autoresearch/ 缺 1（replay.py）
```

例如 `techniques/budget.py` 指向 `research/knowledge/search_space/06_dynamic_budget.md` 与 `research/validation/tests/test_techniques_budget.py` `[e-122]`；`experiments/trace.py` 说输出要靠 `research/autoresearch/replay.py` 校验完整性 `[e-124]`。这些目录在公开树里都不存在。

这条证据把结论从「我没找到」升级为「**作者交付的是裁剪后的树，裁剪口就在这些引用上**」——机制侧（引擎、techniques、bench）完整，判定与知识侧被拿掉了。

9.5 一条并列呈现的冲突 `[c-029]`

论文说输出不逐位相同是正常的、并用 token forcing 统计判定；仓库 README 则说「我们只在逐 token 检查了每个提交的 token 都是目标模型自身的选择之后才接受 final」`[e-104]`。

两种解读都成立：

- 检查发生在**离线流程**里，没进仓库；
- 或者检查根本没做，README 的说法是事后描述。

本归档**不裁决**。能确定的是：能自动跑的判据（统计的）没交付，交付的是一个人工判据的描述。这与 §9.4 的裁剪口一致。

9.6 复现要付出什么

门槛	内容
硬件	1× H100 80GB 或 RTX PRO 6000（六个 setting 分属两种）
环境	Python 3.11 + torch 2.13 + CUDA 13.0，`setup_env.sh` 锁定
权重	`download_models.sh <setting>`，一次一个 setting
数据	`SPECPILOT_DATA`（默认 `~/specpilot_data`），shared/ 存 venv 与 checkpoint
时间	每 setting 6 次运行（stock 2 + final 2 + AR 1，README 口径），每次 30–100 请求

仓库自带 6 个 PNG，其中 `assets/fig_agent.png` 是 README 里那张「一轮循环」示意图——**图的来源是论文，不是可复跑的代码。**

10. 争议与证据缺口

1. **单 agent 对照只有一个 setting、一次运行** `[c-005]`。「成本模型是主要贡献」的因果强度受此限制；它也可能是「这个 setting 恰好需要一个会看权重带宽的 agent」。
2. **ceiling 超过自身估计这件事只有一处** `[c-025]`。2.44× vs 2.12× 是系统推断（论文只并列陈述两个数字），不是作者的结论。

3. **「未交付」的边界** `[c-028]`：本审计只覆盖 commit `78c42af` 的公开快照、只扫非 vendored 文本。作者可能把控制器放在私有分支，也可能从未公开。正确表述是「公开仓库未交付」。
4. **判据的「解释掉」是穷举还是举例**：论文给三种可接受解释，没给完备性论证。
5. **准确率实验的负载单一**：2048 token、五个 benchmark、greedy。更短或更开放的负载下，0.5 点的分辨率未必够。
6. **项目页是 secondary**：只用它交叉核对措辞（晋级门槛、批处理反号），不用它作为任何数字的唯一来源。
7. **未做**：没有在真机上复现任何一个 setting（无 H100 / PRO 6000）；没有读 `python/` 里 SGLang 裁剪副本的 diff（那需要上游对照，超出本专题范围）。

11. 结论：可迁移的是什么

把三条判据工程拆开看 `[c-030]`：

判据	论文怎么做的	为什么可迁移
该往哪看	把上界分解成可重算的因子，每轮取最大者	否则搜索会被最显眼的瓶颈吸住——有对照实验支撑（1.34× vs 2.13×）
算不算改进	只看增益的 95% 下界与噪声之比	点估计在几个百分点的量级上不可用；噪声必须每轮实测
还无不无损	用能同时给出假阴性与假阳性的统计判据	有注入故障实验支撑；「逐位相同」与「准确率」各有系统性失效

这三条都不依赖投机解码。 任何「agent 改系统、系统要保证语义不变」的循环都需要它们：编译器优化、数据库查询改写、数值库替换、量化部署。

反过来说，W12、best-first 树、TLV 是可以替换的产物——论文自己也这么用它们：DSpark 复用三条、PRO 6000 移植两条、EAGLE-3 只复用两条 `[c-024]`。

最后一处值得记住的是**记忆的保留规则**：条目只在「套回 stock 能逐字重建源系统代码」时才保留 `[e-016]`。这把「记忆」从「经验总结」变成了「可执行的补丁集」——否则记忆会退化成不可验证的散文。

12. 产物与验证

```
research/specpilot/
  brief.yaml                    研究简报（7 个子问题，全部 answered）
  sources.csv                   3 个来源（paper / code / page）
  documents.csv                 17 份快照（含 content_hash 与 byte_size）
  evidence.csv                  81 条，十维标注
（code/mech/judge/loop/costmodel/lossless/eval/boundary/memory/cost）
  claims.csv                    33 条（29 高置信 / 3 中 / 1 争议），81 条证据零悬空
  analysis.json                 全部计数，正文数字的唯一来源
  repo_audit.json               仓库规模与组件检索（audit_repo.py 产出）
  dangling_audit.json           悬空引用审计（audit_dangling.py 产出）
  validation.txt                校验器原始输出
  fetch_fulltext.py             arXiv HTML → 纯文本
  fetch_project_page.py         项目页 → 纯文本
```

```
  audit_repo.py                数数文件 / 开关 / setting 规模
  audit_dangling.py            查「引用了但不存在」的路径
  build_specpilot.py           建四张表（quote 全部由 grab() 实取）
  make_analysis.py             汇总计数
  check_numbers.py             核对正文数字与数据是否一致（带反例验证）
  sync_report.py               计数变化时按模板重写正文（不做模糊替换）
  documents/                   17 份快照（论文全文 + 13 份仓库文件 + 项目页 + 2 份审计产
物）
  notes/                       过程笔记
```

检查	结果
`deepresearchrsi.validate`	`[OK]`（见下方机器可读行）
E1 原文子串	`e1_checked=81`、`e1_unverifiable=0`
证据零悬空	`evidence_dangling=[]`（81/81 被引用）
`check_numbers.py`	`[OK]` 14 项计数 + 6 处正文数字 + 校验行全一致（反例：改坏「证据数」立刻报错）
`python tests/test_schemas.py`	本归档两项检查全过；2 项失败在 `pd-disagg`（HEAD 既有，非本次引入，见 `notes/PROCESS.md` §6）

本轮修掉的三个工具缺陷

一、`grab()` 用正则兜白空格，55 条锚点挂了 25 条。 LaTeXML 把 `4.46×` 写成 `4.46 × \times`、`Figure 3` 写成 `Figure\xa0 3`、`'` 写成 `’`。每次失败都得回原文猜措辞。改为**首尾字面锚点 + 去空白匹配 + 索引映射回原文**后，一轮内全部命中，且取出的仍是原文连续区间。

二、`documents.csv` 的 content_hash 与 quote 用了两套文本。 我先按「同形归一」后的文本存 quote，校验器却拿原文比对（它只折叠空白、不做引号映射），于是 20 条 E1 判为「不是原文子串」。修正：**检索可以归一，存储必须原文。**

三、悬空引用审计的正则截断了路径。 `research/scripts/launch.sh` 被截成 `scripts/launch.sh`，误报 2 条缺失。加负向后顾 `(?<![\w/.-])` 后消失——22 条引用、10 条缺失。

三处的共同点是：**当检查器与数据用两套口径时，检查器报的是「我以为的样子」，不是「实际的样子」。**

校验行（机器可读，勿手改）：

(sources=3, evidence=81, claims=33, claims_high=29, claims_disputed=1, c1_violations=0, e1_unverifiable=0)